

TRANSACTION MANAGEMENT

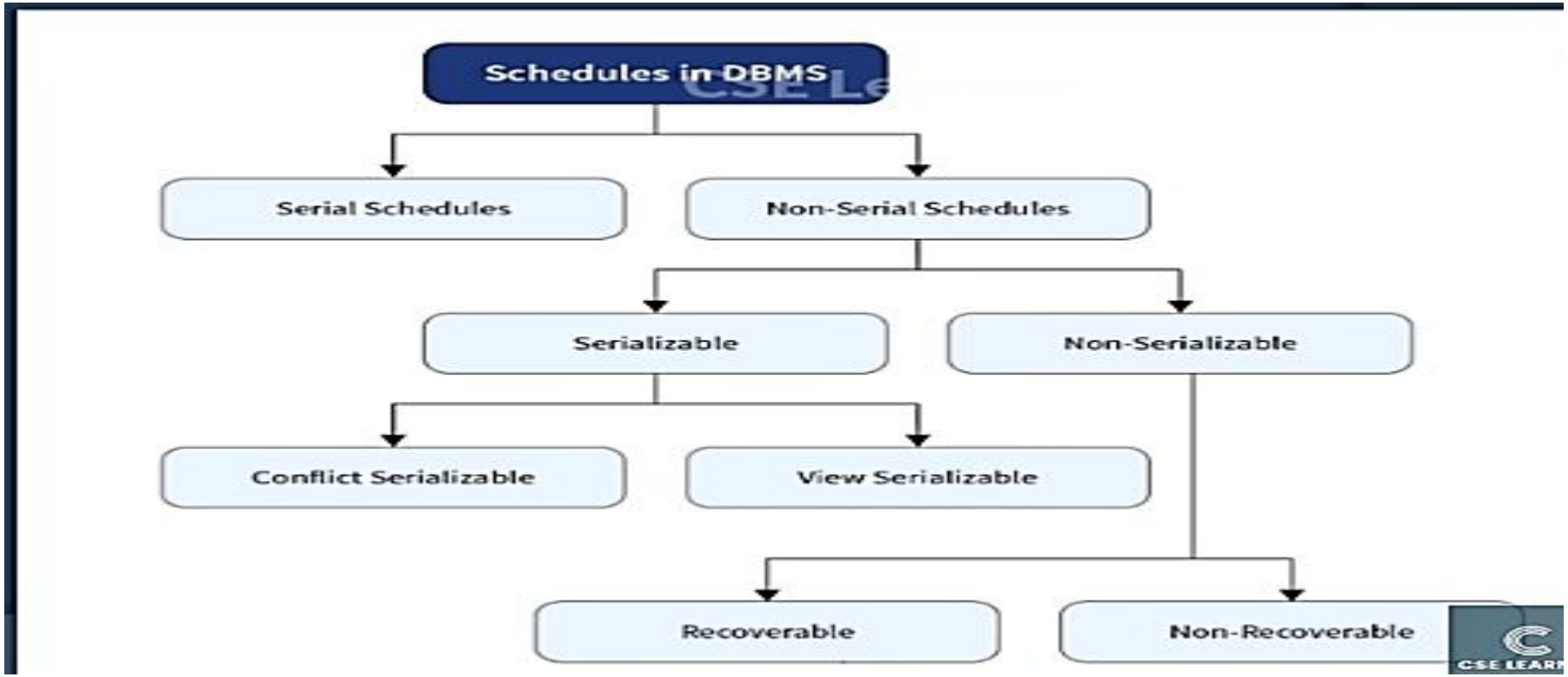


Transactions and Schedules

Introduction

- ✔When multiple transaction requests are made at the same time, we need to decide their order of execution. Thus, a transaction schedule can be defined as a **chronological order of execution of multiple transactions**.
- ✔When there are multiple transactions that are running in a concurrent manner and the order of operation is needed to be set so that **the operations do not overlap each other**, Scheduling is brought into play and the transactions are timed accordingly.

Introduction



Serial Schedule

- The serial schedule is a type of schedule where **one transaction is executed completely before starting another transaction.** In the serial schedule, when the first transaction completes its cycle, then the next transaction is executed.
- Schedules in which the transactions are executed non-interleaved, i.e., a serial schedule is one in which **no transaction starts until a running transaction has ended** are called serial schedules.

T_1	T_2
R(A)	
W(A)	
R(B)	
	W(B)
	R(A)
	R(B)

Serial Schedule

Transaction T1	Transaction T2
R(A)	
W(A)	
R(B)	
W(B)	
commit	
	R(A)
	W(B)
	commit

If n = number of transactions, then a number of serial schedules possible = n!.

3! = 1 * 2 * 3

4! = 1 * 2 * 3 * 4

Therefore, for the above 2 transactions, a total number of serial schedules possible = 2.

Non-Serial Schedule:

- ✔This is a type of Scheduling where the **operations of multiple transactions are interleaved.** This might lead to **a rise in the concurrency problem.** The transactions are **executed in a non-serial manner**, keeping the end result correct and same as the serial schedule. Unlike the serial schedule where one transaction must wait for another to complete all its operation, in the non-serial schedule, **the other transaction proceeds without waiting for the previous transaction to complete.** This sort of schedule does not provide any benefit of the concurrent transaction.
- ✔It can be of two types namely,

1. Serializable and
2. Non-Serializable Schedule.

Non-Serial Schedule:

Non-Serial Schedule :

- In a non-serial Schedule, multiple transactions execute concurrently/simultaneously, unlike the serial Schedule, where one transaction must wait for another to complete all its operations. In the Non-Serial Schedule, the other transaction proceeds without the completion of the previous transaction.

- All the transaction operations are interleaved or mixed with each other.

Non-serial schedules are NOT always recoverable, cascades, strict and consistent.

CSE Learner

Transaction T1	Transaction T2
R1(A)	
W1(A)	
	R2(A)
	W2(A)
R1(B)	
W1(B)	
	R2(B)
	W2(B)

Serializability:

- Serializability is a concept that helps to identify which non-serial schedules are correct and will maintain the consistency of the database. A serializable schedule always leaves the database in a consistent state.

- A non-serial schedule of n transactions is said to be a serializable schedule, if it is equivalent to the serial Schedule of those n transactions.

- A serial schedule does not allow concurrency; only one transaction executes at a time, and the other starts when the already running transaction is finished.

CSE Learner

Serial Schedule	Serializable Schedule
No concurrency allowed	concurrency allowed
Less resource utilization and CPU throughput	Improves both resource utilization and CPU throughput
Transactions executes one after another	Transactions executes concurrently
Less efficient	More efficient

Serializability:

- **Types of Serializability:** Conflict Serializability & View Serializability

Conflicting Operations :

Conflict equivalent

1. T1: Read(A) T2: Read(A)

T1	T2
Read(A)	
	Read(A)

Swapped

T1	T2
	Read(A)
Read(A)	

Schedule S1 Schedule S2

Non-conflicting operations

Non-conflict equivalent

2. T1: Read(A) T2: Write(A)

T1	T2
Read(A)	
	Write(A)

Swapped

T1	T2
	Write(A)
Read(A)	

Schedule S1 Schedule S2

Conflicting operations

The two operations become conflicting if all conditions satisfy:

1. Both belong to separate transactions.
2. They have the same data item.
3. They contain at least one write operation.

Conflict Serializability:

- A schedule is called conflict serializability if after swapping of non-conflicting operations, it can transform into a serial schedule. The schedule will be a conflict serializable if it is conflict equivalent to a serial schedule.

Transaction T1	Transaction T2
Read(A)	
Write(A)	
	Read(A)
	Write(A)
Read(B)	
Write(B)	
	Read(B)
	Write(B)

Before Swapping

Transaction T1	Transaction T2
Read(A)	
Write(A)	
Read(B)	
Write(B)	
	Read(A)
	Write(A)
	Read(B)
	Write(B)

After Swapping

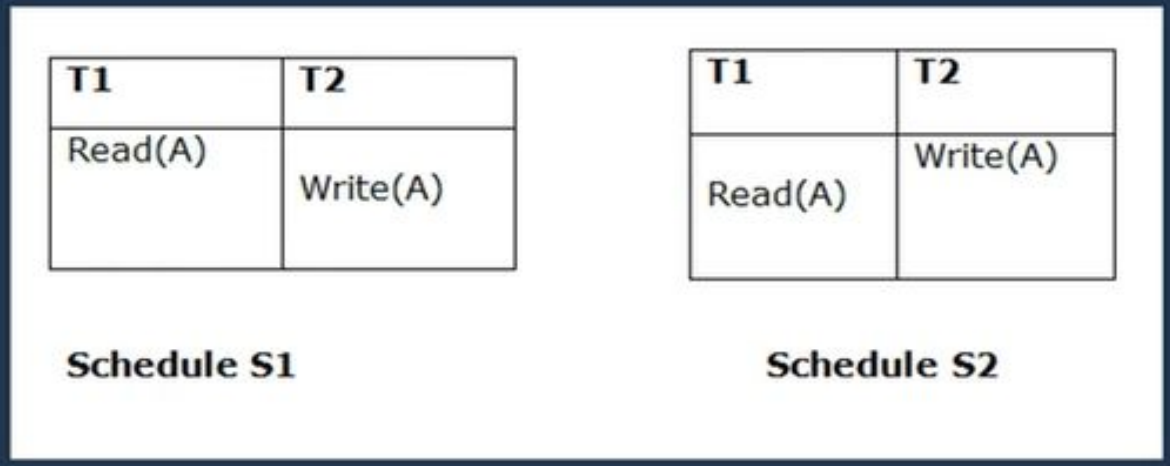
- Thus, by swapping(non-conflicting) 2nd pair of the read/write operation of 'A' data item and 1st pair of the read/write operation of 'B' data item, this non-serial Schedule can be converted into a serializable Schedule. Therefore, *it is conflict serializable*.

View Serializability:

View Equivalent :

- Two schedules S1 and S2 are said to be view equivalent if they satisfy the following conditions :
  1.Initial read 2.Update read 3.Final write

Initial read :



T1	T2
Read(A)	
	Write(A)

Schedule S1

T1	T2
	Write(A)
Read(A)	

Schedule S2

CSE Learner

- An initial read of both schedules must be the same. Suppose two schedule S1 and S2. In schedule S1, if a transaction T1 is reading the data item A, then in S2, transaction T1 should also read A.

View Serializability:

View Equivalent :

- Two schedules S1 and S2 are said to be view equivalent if they satisfy the following conditions :
  1.Initial read 2.Update read 3.Final write

Update read :

T1	T2	T3
Write(A)		
	Write(A)	
		Read(A)

Schedule S1

T1	T2	T3
	Write(A)	
Write(A)		
		Read(A)

Schedule S2

CSE Learner

- Above two schedules are not view equal because, in S1, T3 is reading A updated by T2 and in S2, T3 is reading A updated by T1.

View Serializability:

View Equivalent :

- Two schedules S1 and S2 are said to be view equivalent if they satisfy the following conditions :
  1.Initial read 2.Update read 3.Final write

Final write :

T1	T2	T3
Write(A)	Read(A)	Write(A)

Schedule S1

T1	T2	T3
Write(A)	Read(A)	Write(A)

Schedule S2

CSE Learner

- Above two schedules is view equal because Final write operation in S1 is done by T3 and in S2, the final write operation is also done by T3.

Non - Serializability:

- A non-serial schedule that is not serializable is called a non-serializable schedule. Non-serializable schedules may/may not be consistent or recoverable. Non-serializable schedules may/may not be consistent or recoverable.
- Non-serializable Schedule is divided into types:
    1. Recoverable Schedule
    2. Non-recoverable Schedule

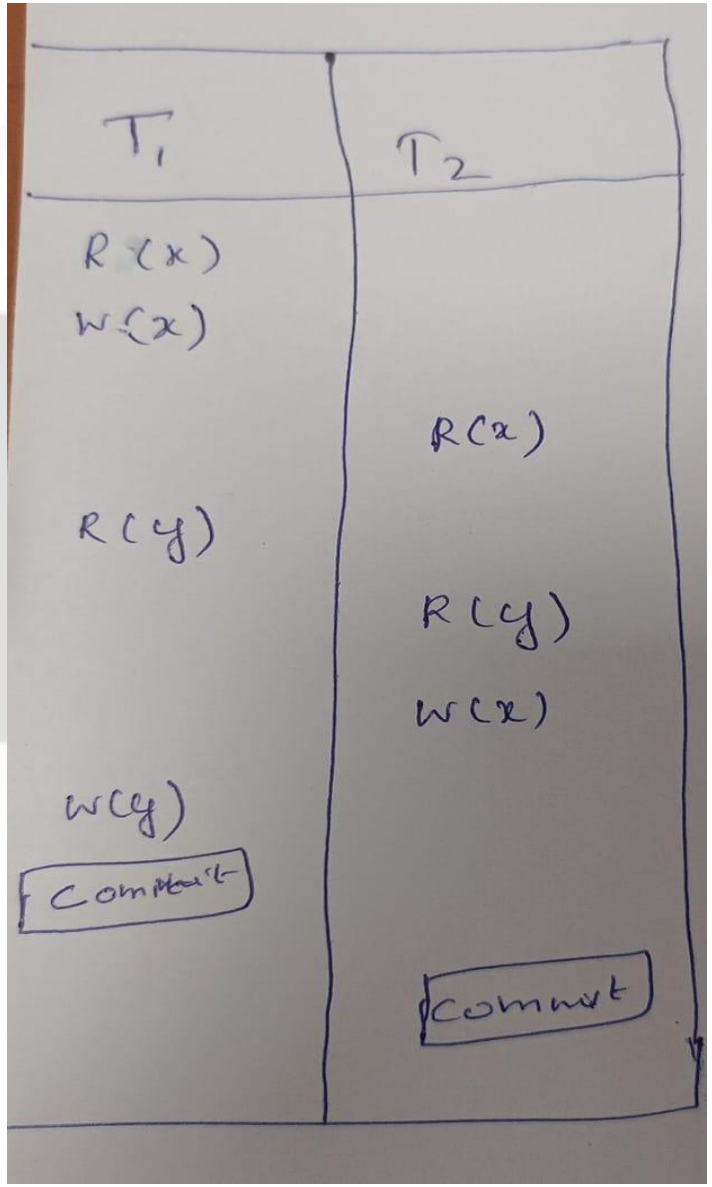
CSE Learner

Recoverable Schedule

Example 1:

S1: R1(x), W1(x), R2(x), R1(y), R2(y),
W2(x), W1(y), C1, C2;

Given schedule follows order of Ti->Tj => C1->C2. Transaction T1 is executed before T2 hence there is no chances of conflict occur. R1(x) appears before W1(x) and transaction T1 is committed before T2 i.e. completion of first transaction performed first update on data item x, hence given schedule is recoverable.



Recoverable Schedule

Example 2: Consider the following schedule involving two transactions T_1 and T_2.

T_1	T_2
R(A)	
W(A)	
	W(A)
	R(A)
commit	
	commit

This is a recoverable schedule since T_1 commits before T_2, that makes the value read by T_2 correct.

Non-Recoverable Schedule

Irrecoverable Schedule: The table below shows a schedule with two transactions, T1 reads and writes A and that value is read and written by T2. T2 commits. But later on, T1 fails. So we have to rollback T1. Since T2 has read the value written by T1, it should also be rollbacked. But we have already committed that. So this schedule is irrecoverable schedule. When Tj is reading the value updated by Ti and Tj is committed before committing of Ti, the schedule will be irrecoverable.

T1	T1's buffer space	T2	T2's Buffer Space	Database
				A=5000
R(A);	A=5000			A=5000
A=A-100;	A=4000			A=5000
W(A);	A=4000			A=4000
		R(A);	A=4000	A=4000
		A=A+500;	A=4500	A=4000
		W(A);	A=4500	A=4500
		Commit;		
Failure Point				
Commit;				

Non-Recoverable Schedule

T1	T1's buffer space	T2	T2's buffer space	Database
				A = 6500
Read(A);	A = 6500			A = 6500
A = A - 500;	A = 6000			A = 6500
Write(A);	A = 6000			A = 6000
		Read(A);	A = 6000	A = 6000
		A =A + 1000;	A = 7000	A = 6000
		Write(A);	A = 7000	A = 7000
Failure Point				
Commit;				
		Commit;		

Non-Recoverable Schedule

T1	T1's buffer space	T2	T2's buffer space	Database
				A = 6500
Read(A);	A = 6500			A = 6500
A = A - 500;	A = 6000			A = 6500
Write(A);	A = 6000			A = 6000
		Read(A);	A = 6000	A = 6000
		A =A + 1000;	A = 7000	A = 6000
		Write(A);	A = 7000	A = 7000
		Commit;		
Failure Point				
Commit;				

Summary

1. Recoverable Schedule :

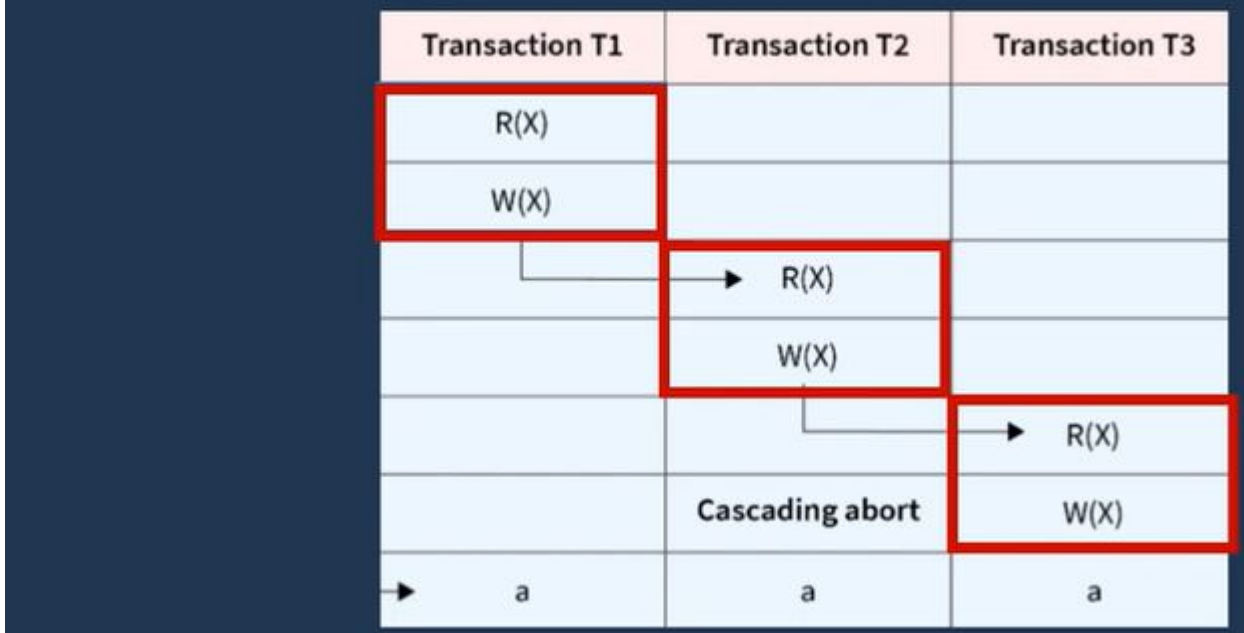
A schedule is recoverable if each *transaction commits only after all the transactions* from which it has read have committed.

2. Non-Recoverable Schedule :

If a *transaction reads the value of an operation from an uncommitted transaction and commits before the transaction from where it has read the value*, then such a schedule is called Non-Recoverable schedule.

Non-Recoverable Schedule

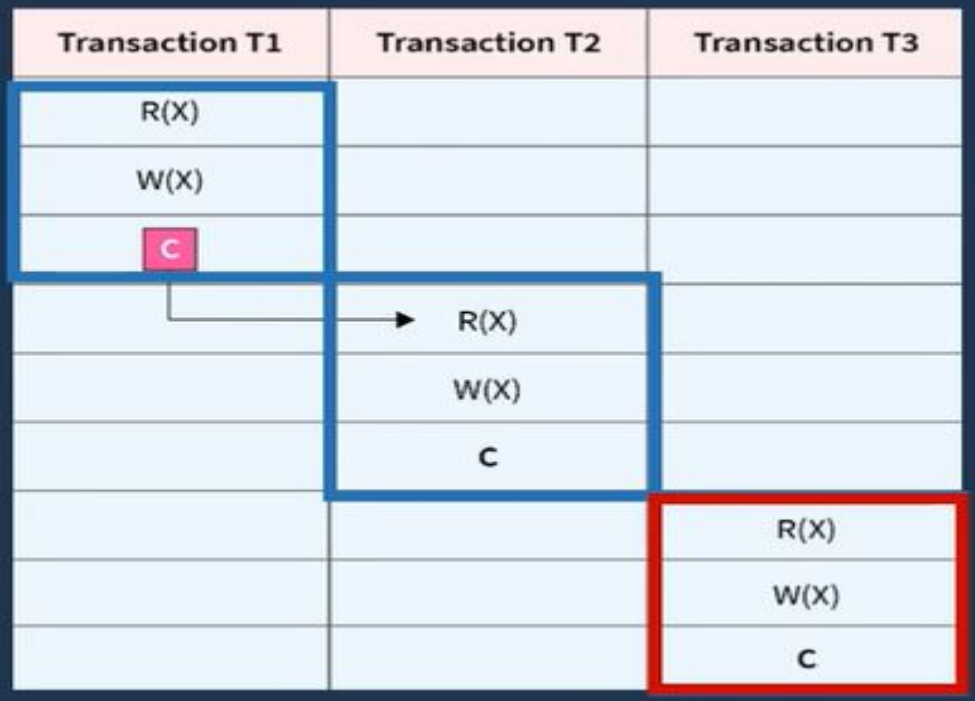
1. Cascading Schedule :



- If in a schedule, *several other dependent transactions are forced to rollback/abort because of the failure of one transaction*, then such a schedule is called a Cascading Schedule or Cascading Rollback or Cascading Abort. It simply leads to the wastage of CPU time.

Non-Recoverable Schedule

2. Cascadeless Schedule :



Transaction T1	Transaction T2	Transaction T3
R(X)		
W(X)		
C		
	R(X)	
	W(X)	
	C	
		R(X)
		W(X)
		C

- If in a schedule, a *transaction is not allowed to read a data item until and unless the last transaction that has been written is committed/aborted*, then such a schedule is called a Cascadeless Schedule. It avoids cascading rollback and thus saves CPU time.